perplexity

I like it!

Great! If you want to move from “nice idea” to “running code”, the next step is to lock down a minimal but complete skeleton: a shared state representation, three observation heads, and a joint training loop.

Below is a compact PyTorch-style blueprint you can adapt. It’s deliberately generic so you can plug in your existing SuperDARN, OMNI, and HF logic.

1. Shared state representation

Assume a 3D ionospheric grid: magnetic latitude × MLT × altitude, with a few channels (e.g., N_e, v_{LOS}, maybe temperature).

```
import torch
import torch.nn as nn

class IonosphereState:
    """
    Latent ionospheric state on a fixed grid.
    state: Tensor of shape [batch, channels, n_lat, n_mlt, n_alt]
    """
    def __init__(self, state: torch.Tensor):
        self.state = state

    @classmethod
    def zeros(cls, batch_size, channels, n_lat, n_mlt, n_alt, device="cpu"):
        state = torch.zeros(
            batch_size, channels, n_lat, n_mlt, n_alt,
            device=device
        )
        return cls(state)
```

You can initialise this from:

- A climatology model.
- The output of an existing empirical model (e.g., IRI, SuperDARN convection model).
- A learned encoder from one of your data streams.

2. Dynamics model (OMNI-driven)

This is your project-2 ML forecaster, reframed as a state transition:

$$\mathbf{x}_{k+1} = f_\theta(\mathbf{x}_k, \mathbf{u}_k)$$

where $\mathbf{u}_k$ are OMNI features (solar wind V, B, coupling functions, etc.).

```
class DynamicsModel(nn.Module):
    """
    Neural state-space dynamics:
      x_{k+1} = f(x_k, u_k)
    x: [B, C, L, M, A]
    u: [B, D_u]  (OMNI features)
    """
    def __init__(self, in_channels, latent_channels, u_dim,
                 n_lat, n_mlt, n_alt, hidden=64):
        super().__init__()
        # Simple ConvNet-in-time style; you can replace with UNet, FNO, etc.
        self.u_embed = nn.Sequential(
            nn.Linear(u_dim, hidden),
            nn.GELU(),
            nn.Linear(hidden, hidden),
        )
        self.conv = nn.Sequential(
            nn.Conv3d(in_channels + 1, hidden, kernel_size=3, padding=1),
            nn.GELU(),
            nn.Conv3d(hidden, hidden, kernel_size=3, padding=1),
            nn.GELU(),
            nn.Conv3d(hidden, latent_channels, kernel_size=3, padding=1),
        )
        self.n_lat = n_lat
        self.mlt = n_mlt
        self.alt = n_alt

    def forward(self, x: torch.Tensor, u: torch.Tensor) -> torch.Tensor:
        # x: [B, C, L, M, A]
        B, C, L, M, A = x.shape
        # Embed drivers
        ue = self.u_embed(u)  # [B, H]
        ue = ue.view(B, 1, 1, 1, 1).expand(-1, -1, L, M, A)  # [B, 1, L, M, A]
        # Concat as extra channel
        xc = torch.cat([x, ue], dim=1)  # [B, C+1, L, M, A]
        dx = self.conv(xc)              # [B, C_lat, L, M, A]
        # Simple Euler step; you can make this more sophisticated
        x_new = x + dx
        return x_new
```

You can swap the Conv3d block for:

- A Fourier Neural Operator / DeepONet block.
- A U-Net over the 3D grid.
- A spectral model in MLT/latitude.

3. Observation heads

3.1 SuperDARN head (project 1)

Map the latent state to expected fitacf-like outputs (e.g., LOS velocity and backscatter power at each range gate/beam).

```
class SuperDARNObservation(nn.Module):
    """
    Maps ionosphere state -> SuperDARN observables.
    For simplicity, assume we predict LOS velocity at each range gate.
    """
    def __init__(self, state_channels, n_beams, n_gates,
                 n_lat, n_mlt, n_alt, hidden=64):
        super().__init__()
        self.n_beams = n_beams
        self.n_gates = n_gates
        # Precomputed geometry: for each (beam, gate), which grid cell(s) contribute
        # Here we use a simple learned mapping; you can inject real geometry.
        self.net = nn.Sequential(
            nn.Conv3d(state_channels, hidden, kernel_size=3, padding=1),
            nn.GELU(),
            nn.Conv3d(hidden, hidden, kernel_size=3, padding=1),
            nn.GELU(),
            nn.Conv3d(hidden, n_beams * n_gates, kernel_size=3, padding=1),
        )
        # Optionally, store a mask or geometry tensor here.

    def forward(self, state: IonosphereState) -> torch.Tensor:
        """
        Returns:
          y_sd: [B, n_beams, n_gates]  (e.g., LOS velocity or power)
        """
        x = state.state  # [B, C, L, M, A]
        out = self.net(x)  # [B, n_beams*n_gates, L, M, A]
        # Global pool over space; replace with geometry-aware pooling in real code
        out = out.mean(dim=(-3, -2, -1))  # [B, n_beams*n_gates]
        out = out.view(-1, self.n_beams, self.n_gates)
        return out
```

In practice, you'd:

- Replace the mean pooling with a geometry-aware projection (e.g., interpolate state to each range gate's virtual location using the standard SuperDARN hv(r) model).[1] [2]
- Output multiple channels (velocity, power, spectral width) if needed.

3.2 HF connectivity / bounce height head (project 3)

Map the state to predicted link existence and reflection heights for each HF path.

```
class HFObservation(nn.Module):
    """
    Maps ionosphere state -> HF observables:
      - link probability (or binary)
      - bounce height estimate
    for each HF path.
    """
    def __init__(self, state_channels, n_paths,
                 n_lat, n_mlt, n_alt, hidden=64):
        super().__init__()
        self.n_paths = n_paths
        self.net = nn.Sequential(
            nn.Conv3d(state_channels, hidden, kernel_size=3, padding=1),
            nn.GELU(),
            nn.Conv3d(hidden, hidden, kernel_size=3, padding=1),
            nn.GELU(),
            nn.Conv3d(hidden, 2 * n_paths, kernel_size=3, padding=1),
        )

    def forward(self, state: IonosphereState):
        """
        Returns:
          link_logit: [B, n_paths]
          height:     [B, n_paths]  (km or grid index)
        """
        x = state.state
        out = self.net(x)  # [B, 2*n_paths, L, M, A]
        out = out.mean(dim=(-3, -2, -1))  # [B, 2*n_paths]
        out = out.view(-1, self.n_paths, 2)
        link_logit = out[..., 0]
        height = out[..., 1]
        return link_logit, height
```

Again, you can replace the mean pooling with a physics-based ray-tracing layer that uses the local $N_e(h)$ profile along each path to compute MUF and reflection height. [3] [4]

3.3 Optional direct ionospheric index head (project 2 targets)

If project 2 currently predicts specific indices (e.g., convection EOF coefficients, TEC, foE), add another head:

```
class IndexObservation(nn.Module):
    def __init__(self, state_channels, n_indices, hidden=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.AdaptiveAvgPool3d(1),
            nn.Flatten(),
            nn.Linear(state_channels, hidden),
            nn.GELU(),
```

```
            nn.Linear(hidden, n_indices),
        )

    def forward(self, state: IonosphereState) -> torch.Tensor:
        x = state.state
        return self.net(x)  # [B, n_indices]
```

4. Joint model and training loop

```
class UnifiedIonosphereModel(nn.Module):
    def __init__(self, dynamics, sd_head, hf_head, index_head=None):
        super().__init__()
        self.dynamics = dynamics
        self.sd_head = sd_head
        self.hf_head = hf_head
        self.index_head = index_head

    def step(self, x: IonosphereState, u: torch.Tensor):
        """One time step: x_{k+1} = f(x_k, u_k)"""
        x_new_state = self.dynamics(x.state, u)
        return IonosphereState(x_new_state)

    def observe_sd(self, x: IonosphereState):
        return self.sd_head(x)

    def observe_hf(self, x: IonosphereState):
        return self.hf_head(x)

    def observe_indices(self, x: IonosphereState):
        if self.index_head is None:
            return None
        return self.index_head(x)
```

Example training step

Assume you have a batch of trajectories:

- `x0`: initial state (maybe from climatology or encoder).
- `u[k]`: OMNI drivers at each time.
- `y_sd[k]`: SuperDARN observations.
- `y_hf_link[k]`, `y_hf_height[k]`: HF observations.
- Optionally `y_idx[k]`: index targets.

```
def train_step(model, x0, us, y_sd_list, y_hf_link_list, y_hf_height_list,
               optimizer, device="cpu"):
    """
    us: list[Tensor] of length T, each [B, D_u]
    y_sd_list: list[Tensor] of length T, each [B, n_beams, n_gates]
    y_hf_link_list: list[Tensor] of length T, each [B, n_paths]
    y_hf_height_list: list[Tensor] of length T, each [B, n_paths]
    """
```

```
    model.train()
    optimizer.zero_grad()

    x = x0
    total_loss = 0.0

    for k, u in enumerate(us):
        # Predict next state
        x = model.step(x, u)

        # Observations
        y_sd_pred = model.observe_sd(x)       # [B, n_beams, n_gates]
        link_logit, height_pred = model.observe_hf(x)  # [B, n_paths], [B, n_paths]

        # Losses
        y_sd_true = y_sd_list[k]
        y_hf_link_true = y_hf_link_list[k]
        y_hf_height_true = y_hf_height_list[k]

        loss_sd = ((y_sd_pred - y_sd_true)**2).mean()

        # Binary cross-entropy for link existence
        loss_link = nn.functional.binary_cross_entropy_with_logits(
            link_logit, y_hf_link_true.float()
        )

        # MSE for bounce height (mask out invalid links if needed)
        loss_height = ((height_pred - y_hf_height_true)**2).mean()

        # Weighted sum
        loss = loss_sd + 0.5 * loss_link + 0.5 * loss_height
        # + optionally physics-based regularisers here

        loss.backward()
        total_loss += loss.item()

    optimizer.step()
    return total_loss / len(us)
```

You can extend this to:

- Multiple time steps with teacher forcing.
- Variational / ensemble methods for uncertainty.
- Physics-informed losses (e.g., penalise non-smooth fields, enforce approximate continuity). [5] [6]

5. How to plug in your existing code

- **Project 1 (SuperDARN):**
  - Use your current RST/fitacf pipeline to generate `y_sd_true`.
  - Optionally precompute a geometry mapping from grid cells to range gates and inject it into `SuperDARNObservation`.

- **Project 2 (OMNI + ML):**
  - Wrap your existing forecaster as `DynamicsModel` (or distil it into this architecture).
  - Use its current targets as `IndexObservation` if you want to keep those outputs.
- **Project 3 (HF network):**
  - Use your existing link/height inference to generate `y_hf_link_true` and `y_hf_height_true`.
  - Replace or augment the simple net in `HFObservation` with your current propagation model, made differentiable.

If you tell me your preferred grid sizes, state variables, and what your current models output (shapes, units), I can specialise this into a near-drop-in skeleton with concrete dimensions and example data loaders.

⁂

1. https://nora.nerc.ac.uk/id/eprint/3087/1/Chisham_AG07228.pdf
2. https://angeo.copernicus.org/articles/26/823/2008/
3. https://www.nature.com/articles/s41598-025-87930-8
4. https://www.qsl.net/4x4xm/HF-Propagation.htm
5. https://eos.org/editors-vox/machine-learning-helps-to-solve-problems-in-heliophysics
6. https://arxiv.org/html/2509.19160v1